

GitScout

Software Requirements Specification

SRS 1.0

Group 7

Submission Date: February 24, 2026

Team Members

Xiangan He (Project Manager)
Khushi Sridhar (Frontend Developer)
Haomiao Hu (Frontend / UX Designer)
Bowang Lang (Backend Developer)
Cong Chen (Backend Developer)
Joseph Ma (QA/Testing Lead)

Repository: github.com/xors-software/git-scout

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions and Acronyms
- 1.4 References

2. Overall Description

- 2.1 Business Goals and Value
- 2.2 Product Perspective
- 2.2 User Classes and Characteristics
- 2.3 Operating Environment
- 2.4 Design and Implementation Constraints
- 2.5 Assumptions and Dependencies

3. Functional Requirements

- 3.1 FR1: Candidate Search
- 3.2 FR2: Candidate Profile Analytics
- 3.3 FR3: Talent Scoring Algorithm
- 3.4 FR4: Shortlists
- 3.5 FR5: Export

4. Non-Functional Requirements

- 4.1 Performance
- 4.2 Reliability and Availability
- 4.3 Usability
- 4.4 Security
- 4.5 Maintainability

5. User Stories

- 5.1 Recruiter Stories
- 5.2 Hiring Manager Stories

6. External Interface Requirements

- 6.1 User Interface
- 6.2 Software Interfaces
- 6.3 Communication Interfaces

7. System Requirements and Constraints

- 7.1 Development Environment
- 7.2 Deployment
- 7.3 Assumptions

8. Requirements Traceability

9. Appendix

- 9.1 Revision History
- 9.2 Related Documents

10. Team Member Contributions Since Last Submission

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for GitScout, a web-based talent sourcing platform. The document serves as the authoritative reference for all system behavior expected in the Minimum Viable Product (MVP) and provides traceability from user needs to implementation. It is intended for the development team, QA, and project stakeholders.

1.2 Scope

GitScout analyzes publicly available GitHub profile and activity data to help recruiters and hiring managers identify, evaluate, and organize developer candidates. The MVP delivers the following capabilities:

- Advanced search and filtering over GitHub-derived candidate attributes (language, activity, location, followers).
- Candidate profile analytics dashboard with activity trends and summary metrics.
- Transparent, versioned talent scoring algorithm (v1).
- Candidate shortlists: save, organize, and compare candidates.
- Export shortlisted candidates to CSV/PDF.

Out of scope for this phase: direct outreach/messaging integrations, LinkedIn or other external professional network integrations, and AI-powered job-description-to-candidate matching.

1.3 Definitions and Acronyms

Term	Definition
Candidate	A GitHub user whose public profile data is indexed and scored by GitScout.
Shortlist	A named, user-created collection of candidates saved for further review or export.
Talent Score	A composite numeric score derived from a candidate's GitHub activity, contributions, and profile attributes.
SCI	Software Configuration Item — any artifact under version control (code, docs, tests, configs).
MVP	Minimum Viable Product — the initial release scope.
CI	Continuous Integration.

1.4 References

- GitScout SCMP (Software Configuration Management Plan), v1.0, 2026-02-10.
- GitScout SPMP (Software Project Management Plan), v1.0, 2026-02-17.
- GitHub REST API documentation: <https://docs.github.com/en/rest>

- IEEE 830-1998: Recommended Practice for Software Requirements Specifications.

2. Overall Description

2.1 Business Goals and Value

GitScout aims to assist recruiters and hiring teams in efficiently identifying potential technical candidates using publicly available GitHub data. The system helps reduce the time required for manual profile searching by providing structured search, filtering, and candidate insights in a single interface. By organizing GitHub information into searchable results, the platform supports faster candidate discovery and preliminary evaluation. The tool is intended to keep the process simple, transparent, and scalable for small recruitment teams or academic hiring scenarios.

2.2 Product Perspective

GitScout is a standalone web application. It interfaces with the GitHub REST API as its primary external data source. The system does not replace or extend an existing product. It consists of a Next.js frontend, an Elysia (Bun) backend, and a SQLite database for local persistence of shortlists and cached profile data. Deployment targets Railway or equivalent managed hosting.

2.3 User Classes and Characteristics

User Class	Description	Technical Level
Recruiter	Non-technical hiring professional searching for developer talent.	Low. Needs intuitive UI, plain-language metrics, and guided workflows.
Hiring Manager	Technical lead or engineering manager evaluating candidates.	Moderate to high. Comfortable with code-level metrics and scoring details.

2.4 Operating Environment

- Client: Modern web browsers (Chrome, Firefox, Safari, Edge; latest two major versions).
- Server: Node.js (Bun runtime), deployed on Railway or equivalent PaaS.
- Database: SQLite for shortlist persistence and profile caching.
- External dependency: GitHub REST API (authenticated via personal access token or OAuth app).

2.5 Design and Implementation Constraints

- GitHub API rate limits: 5,000 requests/hour for authenticated requests. Caching and backoff strategies are required.
- All data sourced from GitHub must be publicly available; no private repository or private profile data is accessed.
- The scoring algorithm must be transparent and versioned so users can understand how scores are derived.
- The tech stack is fixed: Next.js, Elysia/Bun, TypeScript, SQLite.

2.6 Assumptions and Dependencies

- GitHub REST API remains available and backward-compatible for the endpoints used.
- Candidate data is limited to what GitHub exposes publicly; profile completeness varies.
- The team operates under a 2-week sprint cadence as defined in the SCMP/SPMP.
- SQLite is sufficient for MVP-scale data; migration to PostgreSQL may be considered post-MVP.

3. Functional Requirements

3.1 FR1: Candidate Search

The system shall allow users to search for GitHub users and filter results based on publicly available attributes.

ID	Requirement	Priority
FR1.1	The system shall accept a search query containing one or more of: username keyword, programming language, location, minimum followers count, and minimum public repository count.	Must
FR1.2	The system shall return paginated search results (default 20 per page) from the GitHub Search API.	Must
FR1.3	The system shall allow sorting of search results by talent score, followers, repositories, or recent activity.	Must
FR1.4	The system shall display a summary card for each result showing avatar, username, bio, primary languages, and talent score.	Must
FR1.5	Search results shall be available within 3 seconds for a typical query under normal API conditions.	Should

3.2 FR2: Candidate Profile Analytics

The system shall provide a detailed analytics view for individual candidates.

ID	Requirement	Priority
FR2.1	The system shall display a candidate profile page with GitHub-sourced data: avatar, name, bio, company, location, public repos, followers, and following counts.	Must
FR2.2	The system shall display a contribution activity chart showing commit frequency over the past 12 months.	Must
FR2.3	The system shall display a breakdown of programming languages used across public repositories.	Must
FR2.4	The system shall display the candidate's talent score with a breakdown of contributing factors.	Must
FR2.5	The system shall show the candidate's most recently updated public repositories with star and fork counts.	Should

3.3 FR3: Talent Scoring Algorithm

The system shall compute a transparent, versioned talent score for each candidate.

ID	Requirement	Priority
----	-------------	----------

FR3.1	The scoring algorithm shall produce a numeric score (0–100) based on weighted factors: commit activity, repository diversity, follower count, contribution recency, and language breadth.	Must
FR3.2	The scoring algorithm version (e.g., v1) shall be displayed alongside every score.	Must
FR3.3	The system shall expose a score breakdown showing the contribution of each factor to the total.	Must
FR3.4	The scoring logic shall reside in a shared package (/packages/scoring) to ensure frontend and backend consistency.	Must

3.4 FR4: Shortlists

The system shall allow users to create and manage named collections of candidates.

ID	Requirement	Priority
FR4.1	The system shall allow users to create a named shortlist.	Must
FR4.2	The system shall allow users to add candidates to a shortlist from search results or a profile page.	Must
FR4.3	The system shall allow users to remove candidates from a shortlist.	Must
FR4.4	The system shall display shortlists with candidate summary cards and support sorting by talent score or date added.	Must
FR4.5	The system shall persist shortlists in the database across sessions.	Must
FR4.6	The system shall allow users to compare two or more shortlisted candidates side-by-side using name, username, score, top languages, followers, and repos.	Should

3.5 FR5: Export

The system shall allow users to export shortlisted candidates.

ID	Requirement	Priority
FR5.1	The system shall export a shortlist to CSV format containing: username, name, talent score, top languages, location, and profile URL.	Must
FR5.2	The system shall export a shortlist to PDF format with a formatted summary report. PDF shall include a title, export date, shortlist name, and a table with the same fields as FR5.1	Should
FR5.3	The export file shall be downloadable via the browser.	Must

4. Non-Functional Requirements

4.1 Performance

ID	Requirement	Priority
NFR1.1	Page load time (initial) shall be under 2 seconds on a broadband connection.	Must
NFR1.2	Search API response time shall be under 3 seconds for 95% of requests under normal GitHub API conditions.	Must
NFR1.3	Profile page rendering shall complete within 2 seconds after API data is received.	Should
NFR1.4	Export generation (CSV) for a shortlist of up to 50 candidates shall complete within 5 seconds.	Should

4.2 Reliability and Availability

ID	Requirement	Priority
NFR2.1	The system shall handle GitHub API rate-limit responses (HTTP 403/429) gracefully by displaying a user-facing message with estimated retry time.	Must
NFR2.2	The system shall cache previously fetched candidate profiles for at least 1 hour to reduce redundant API calls.	Must
NFR2.3	The system shall not lose shortlist data due to server restarts (data persisted in SQLite).	Must

4.3 Usability

ID	Requirement	Priority
NFR3.1	The interface shall be responsive and functional on viewport widths from 1024px to 1920px.	Must
NFR3.2	All interactive elements shall be accessible via keyboard navigation.	Should
NFR3.3	Search, shortlist, and export workflows shall each be completable in three or fewer clicks from the dashboard.	Should

4.4 Security

ID	Requirement	Priority
NFR4.1	GitHub API tokens shall be stored server-side and never exposed to the client.	Must

NFR4.2	All API endpoints shall validate and sanitize input parameters.	Must
NFR4.3	The application shall not store or transmit any data beyond what is publicly available on GitHub.	Must

4.5 Maintainability

ID	Requirement	Priority
NFR5.1	The codebase shall enforce TypeScript strict mode with no implicit any.	Must
NFR5.2	CI pipeline shall enforce linting, type-checking, and unit tests on every pull request.	Must
NFR5.3	The scoring algorithm shall be isolated in a shared package to allow independent versioning and testing.	Must

5. User Stories

5.1 Recruiter Stories

ID	User Story	Acceptance Criteria
US-R-001	As a recruiter, I want to search for developers by programming language and location so I can find candidates relevant to my open role.	Results are filtered correctly; at least language and location filters work.
US-R-002	As a recruiter, I want to see a talent score for each candidate so I can quickly gauge their activity and skill level.	Score (0–100) is visible on search result cards and profile pages.
US-R-003	As a recruiter, I want to save candidates to a shortlist so I can review them later.	Candidate appears in the shortlist after clicking Add; persists across sessions.
US-R-004	As a recruiter, I want to export my shortlist to CSV so I can share it with my team via email or spreadsheet.	CSV downloads with correct candidate data.
US-R-005	As a recruiter, I want the interface to use plain language and clear visuals so I don't need technical expertise to evaluate candidates.	Non-technical metrics are labeled clearly; no raw JSON or code exposed in the UI.

5.2 Hiring Manager Stories

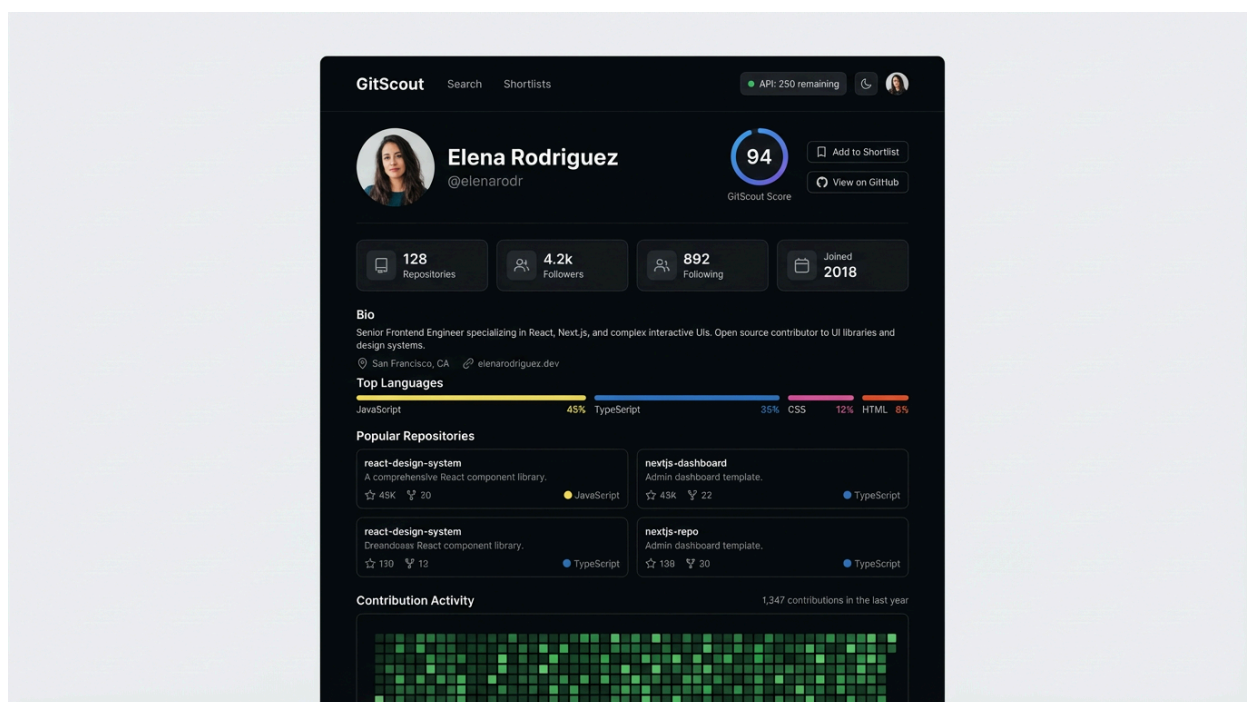
ID	User Story	Acceptance Criteria
US-H-001	As a hiring manager, I want to view a candidate's contribution history chart so I can assess consistency and recency of their work.	12-month activity chart renders on profile page.
US-H-002	As a hiring manager, I want to see the scoring algorithm breakdown so I can understand what factors drive a candidate's score.	Score breakdown with factor weights is visible on profile page.
US-H-003	As a hiring manager, I want to compare two or more candidates side-by-side so I can make informed hiring decisions.	Comparison view shows selected candidates with aligned metrics.
US-H-004	As a hiring manager, I want to sort search results by talent score so I can prioritize the strongest candidates.	Results reorder correctly when sorting by score.

6. External Interface Requirements

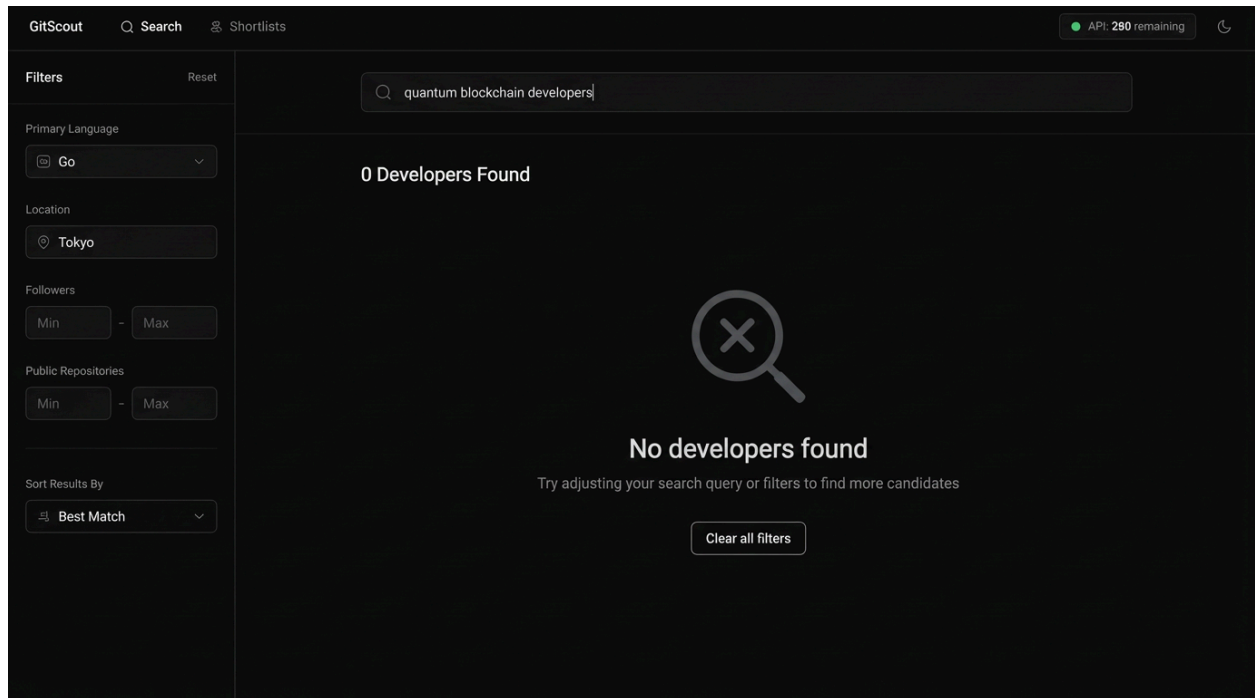
6.1 User Interface Design

The web interface shall consist of the following primary views:

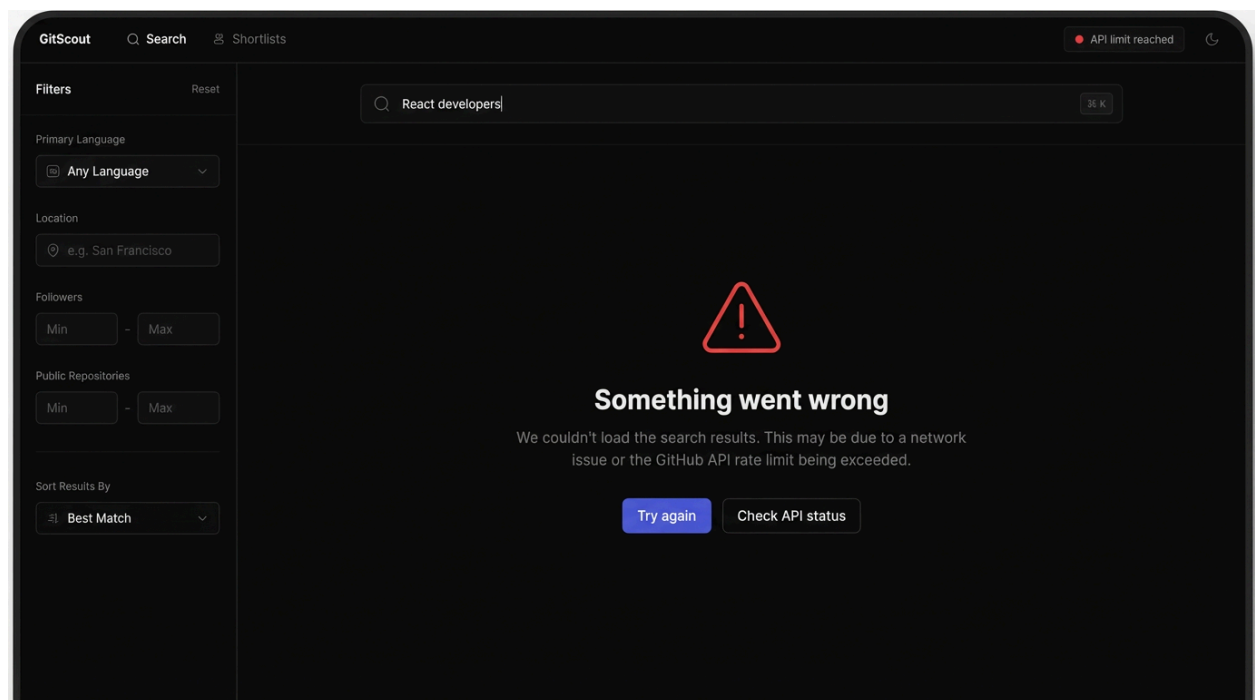
- Dashboard / Search: Landing page with search bar, filters, and paginated results.
- Candidate Profile: Detailed analytics view with activity charts, language breakdown, score breakdown, and an action to add to shortlist.
- Shortlists: List of user-created shortlists with candidate summary cards, sorting, removal, and export actions.
- Comparison View: Side-by-side display of selected candidates.



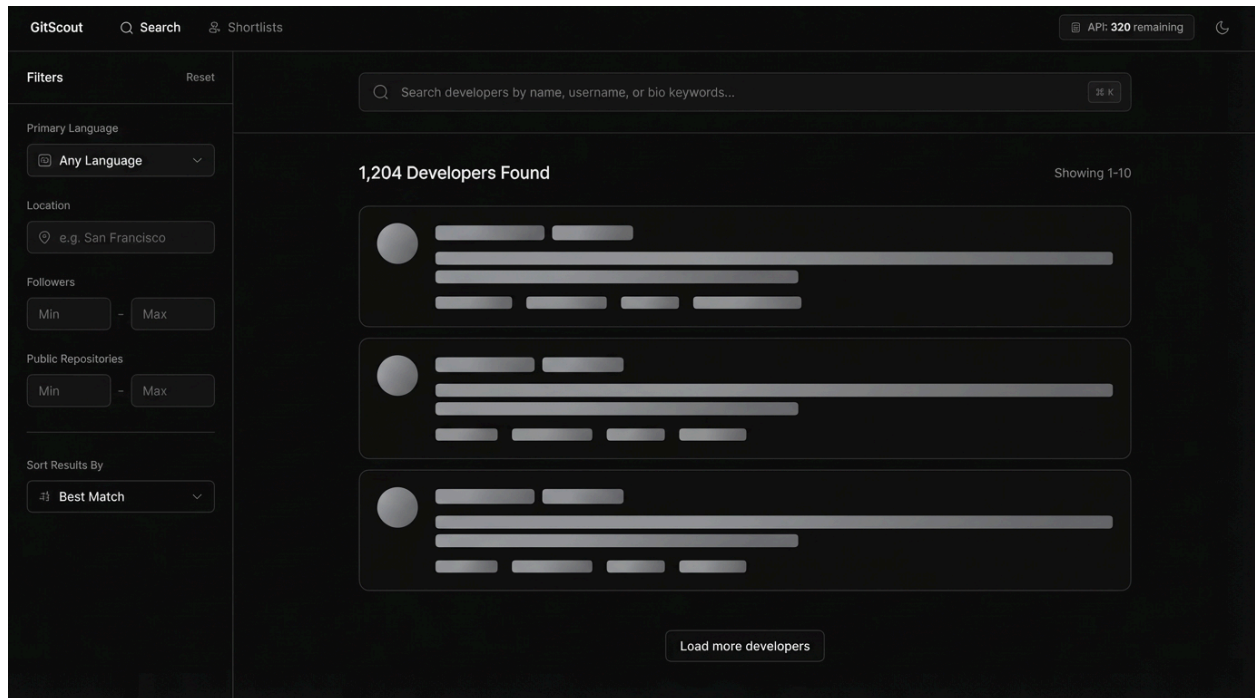
Dev profile



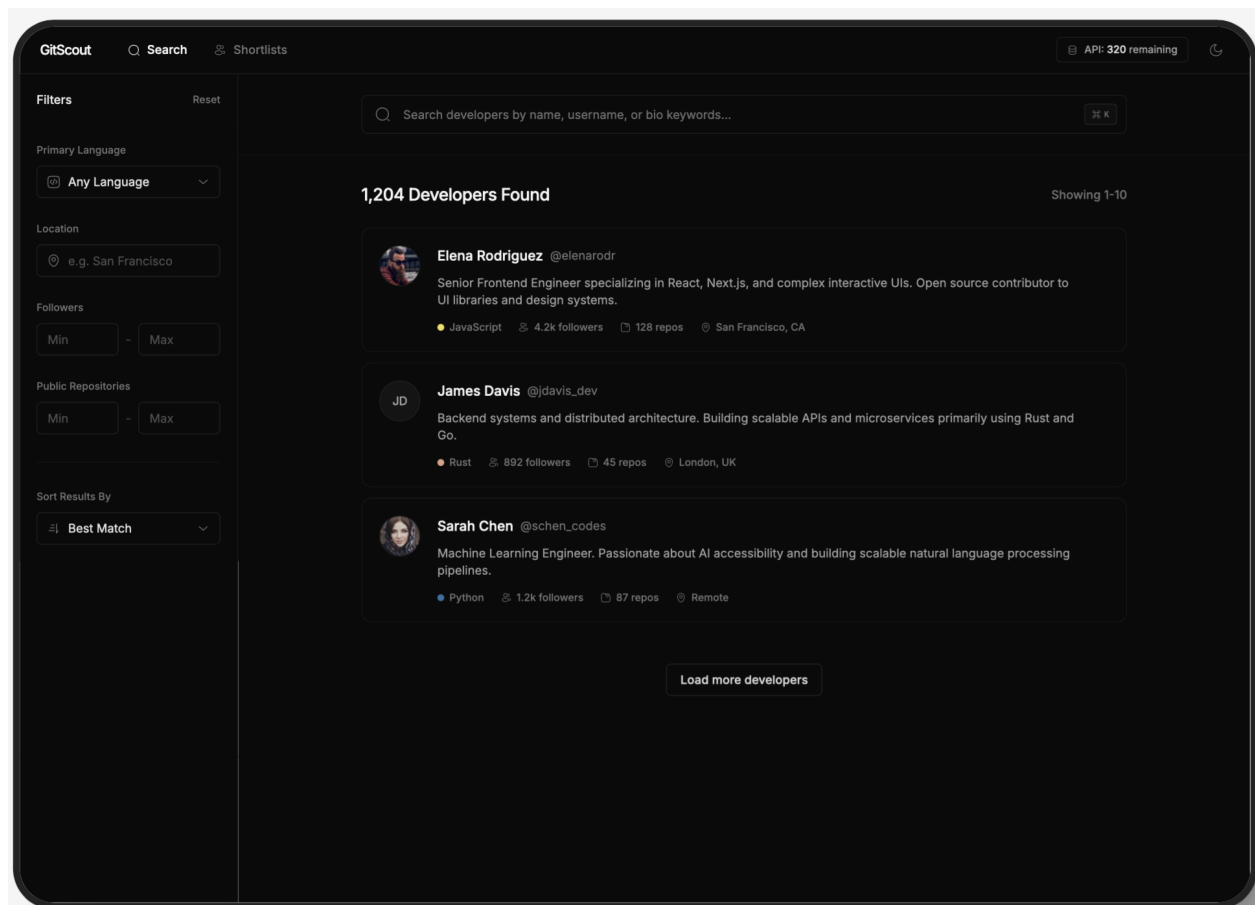
Empty State



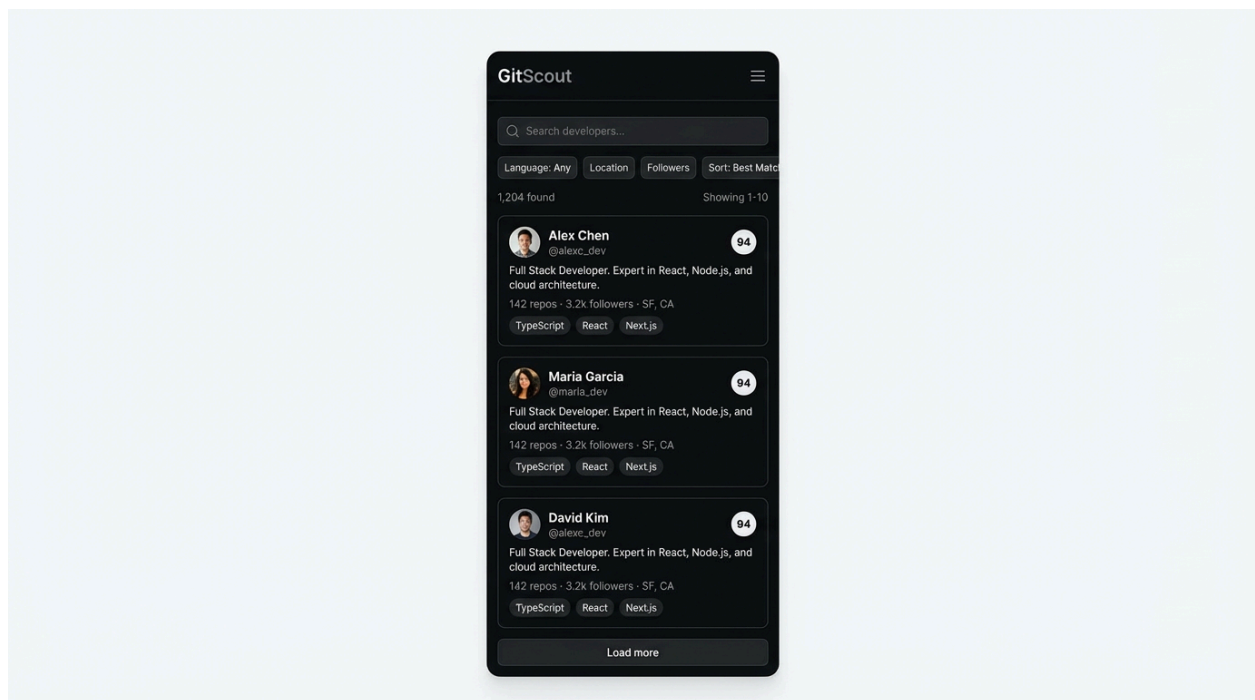
Error state



Loading skeleton



Full page layout



Mobile responsive view

6.2 Software Interfaces

Interface	Details
GitHub REST API	Primary data source. Endpoints used: /search/users, /users/{username}, /users/{username}/repos, /users/{username}/events. Authenticated via server-side token.
SQLite Database	Local persistence for shortlists (shortlist, shortlist_candidates tables) and profile cache (profile_cache table).

6.3 Communication Interfaces

All client-server communication uses HTTPS. The frontend communicates with the backend via a JSON REST API served by the Elysia framework. The backend communicates with the GitHub API over HTTPS.

7. System Requirements and Constraints

7.1 Development Environment

- Runtime: Bun (latest stable) for backend; Node.js 20+ for frontend tooling.
- Language: TypeScript (strict mode) across all packages.
- Frontend framework: Next.js 14+.
- Backend framework: Elysia (Bun-native HTTP framework).
- Testing: Vitest for unit/integration tests.
- CI: GitHub Actions (lint, typecheck, unit tests, smoke tests).

7.2 Deployment

- Hosting: Railway or equivalent PaaS with support for Bun.
- Database: SQLite file on persistent volume.
- Environment variables: GitHub API token, server port, and database path managed via environment configuration.

7.3 Assumptions

The system assumes users have a modern browser with JavaScript enabled. The system assumes GitHub API availability and does not provide offline functionality. The initial deployment targets a single-tenant use case; multi-tenancy and user authentication are deferred to post-MVP.

8. Requirements Traceability

The following matrix maps user stories to functional requirements to support verification and validation planning.

User Story	Functional Req(s)	Verification
US-R-001	FR1.1, FR1.2, FR1.4	Integration test: search with language + location filter returns filtered results.
US-R-002	FR3.1, FR3.2	Unit test: scoring algorithm returns 0–100 with version label.
US-R-003	FR4.1, FR4.2, FR4.5	Integration test: add candidate to shortlist; verify persistence after restart.
US-R-004	FR5.1, FR5.3	System test: export shortlist and verify CSV content.
US-H-001	FR2.2	UI test: activity chart renders 12-month data.
US-H-002	FR3.3	UI test: score breakdown panel is visible and sums to total.
US-H-003	FR4.6	UI test: comparison view shows aligned metrics for 2+ candidates.

9. Appendix

9.1 Revision History

Date	Version	Author	Description
2026-02-21	1.0	Xiangan He	Initial SRS for GitScout MVP.

9.2 Related Documents

- SCMP: Defines SCM policies, branching model, CI gates, and artifact management.
- SPMP: Defines team roles, sprint cadence, risk management, and project scheduling.
- SDD (forthcoming): Will define architecture, API contracts, UI component design, and database schema in detail.

10. Team Member Contributions since Last Submission

Xiangan He: Authored the initial draft of the Software Requirements Specification (SRS), defining functional and non-functional requirements, user stories, and the traceability matrix for the GitScout MVP. Reviewed and approved team pull requests to maintain code quality and SCM compliance. Collaborated with Haomiao on design deliverables. Prototyping efforts are taking place in this [PR](#).

Khushi Sridhar: Built the GitScout search page UI with a search bar, filters, and results section, replacing the initial starter page. Connected the interface to the existing search hooks and verified loading, empty, and error states. This establishes the basic frontend structure for browsing candidate profiles. Additionally, reviewed the SRS document and made updates by refining requirements, adding the Business Goals section, and including user stories to better align the document with the project scope.

PR Link: <https://github.com/xors-software/git-scout/pull/22>

Bowang Lang: Engineered and deployed the Candidate Profile route plugin within `server/src/routes/candidates` using the Elysia framework. Implemented parallel data fetching from GitHub services (Profile, Repos, and Events) using `Promise.all` to optimize performance and minimize endpoint latency. Designed and integrated a comprehensive scoring algorithm that evaluates candidates based on popularity, activity, and consistency metrics. Developed automated logic for language distribution breakdown and star-based repository ranking to provide deep technical insights.

PR Link: <https://github.com/xors-software/git-scout/pull/24>

Cong Chen: Implemented a SQLite-backed shortlist and candidate management module using Elysia. Designed database schema initialization with foreign key constraints, built CRUD APIs for shortlists and candidate associations, and ensured data integrity with proper timestamp handling. Resolved NOT NULL constraint issues and validated all endpoints via Swagger.

PR Link: <https://github.com/xors-software/git-scout/pull/18>

Haomiao Hu: Designed the implementation-ready UI specifications for the GitScout global navigation and search experience. This includes the global top navigation bar with API rate limit indicator, as well as concrete screen layouts for the search page and results grid. Produced detailed UI notes documenting layout structure, spacing, alignment, and component positioning to eliminate ambiguity for frontend implementation. Contributed implementation-ready screen mockups and documentation referenced in the corresponding UI pull request.

PR Link: <https://github.com/xors-software/git-scout/pull/21>

Joseph Ma: This week I focused on improving the backend testing setup and making the server easier to work with. I added initial smoke tests for the health, search, and candidates endpoints using Bun and Elysia's built-in testing utilities, and refactored the server bootstrap so the app can be imported safely in tests without starting a live server. I kept the changes incremental and scoped to make reviews easier. As a next step, I plan to finish the remaining shortlist and export endpoint tests to complete the server-side test coverage

PR Link: <https://github.com/xors-software/git-scout/pull/17>